

NAGARYATRA
(A WEB-BASED APPLICATION)

BY
Shibu Sharma
(TU Reg. No. 7-2-1073-25-2021)
(College Roll No. 13964/21)
Himalaya Darshan College

A Project Report Submitted to
Faculty of Management, Tribhuvan University
in partial fulfillment of the requirements for the degree of

Bachelor of Information Management (BIM)

Biratnagar
June, 2025

STUDENT DECLARATION

This is to certify that I have completed the Project entitled “NagarYatra” under the guidance of “Mr. Nabin Ghimire” in partial fulfillment of the requirements for the degree of **Bachelor of Information Management** at Faculty of Management, Tribhuvan University. This is my original work and we have not submitted it earlier elsewhere.

Date: 3rd June,2025

Signature:

Name: Shibu Sharma

TU Reg No. 7-2-1073-25-2021

CERTIFICATE FROM THE SUPERVISOR

This is to certify that the project entitled “NagarYatra” is an academic work done by “Shibu Sharma” submitted in the partial fulfillment of the requirements for the degree of **Bachelor of Information Management** at Faculty of Management, Tribhuvan University under my guidance and supervision. To the best of my knowledge, the information presented by them in the project report has not been submitted earlier.

Signature of the Supervisor

Name: Mr. Nabin Ghimire

Designation: Lecturer

Date: 3rd June, 2025

APPROVAL SHEET

This is to certify that the project titled **NagarYatra** submitted **Shibu Sharma** has been examined and approved. In our opinion, it meets the required scope and quality standards for a project submitted in partial fulfillment of the requirements for the degree of Bachelor of Information Management (BIM).

Approval Panel:

S.No.	Name	Designation	Signature
1	Mr. Nabin Ghimire	Project Supervisor	
2	Er. Sumit Kumar Shah	Program Coordinator	
3	Mr. Kushal Niroula	Internal Examiner	
4	Mr. Harendra Subedi	External Examiner	

Date of Defense: 3rd June,2025
Department: Department of IT
Faculty: Faculty of Management

ACKNOWLEDGEMENT

The success of this project required guidance and supervision. I would like to express my sincere gratitude to Project Supervisor **Mr. Nabin Ghimire** for his expert guidance and continuous encouragement throughout the project documentation and website development.

I also want to thank, Head of IT Department, **Er. Sumit Shah** for creating the required academic environment which made my task appreciable and **Mr. Kushal Niroula**, coordinator for his cordial support, valuable information, and guidance, which helped me in completing this task through various stages.

I would like to express my sincere gratitude to my project partner, **Kajal Kumari Mehta**, for her valuable contributions in designing the website forms and landing page, as well as for creating all the project diagrams with remarkable clarity and efficiency. Her dedication and attention to detail played a vital role in the success of this project.

I perceive as this opportunity as a big milestone as a career development. I will strive to use gained knowledge in the best possible way, and work on further improvement, to attain desired career objectives. Hope to co-operation with all of you in the future.

I would like to acknowledge the direct and indirect contributions of all individuals who supported me at various stages of this project. Their assistance and encouragement played a vital role in the successful completion of this work. I would like to thank all of them for their invaluable support and guidance.

Name: Shibu Sharma
TU Reg. No.: 7-2-1073-25-2021

ABSTRACT

Urban transportation often faces challenges such as inefficiency, lack of cost transparency, and operational delays, making it difficult for citizens to access reliable transport services. NagarYatra is a web-based application developed using HTML, CSS, JavaScript, PHP, and MySQL, designed to streamline the booking process for city transport options, including E-Safari, bikes, autos, and other vehicles. The platform allows users to book rides for short trips, pre-book rides for special events, ensuring a secure, transparent, and user-friendly experience. By integrating real-time booking, automated fare calculation, and ride management features, NagarYatra improves operational efficiency for drivers while offering convenience to passengers. Additionally, the system enhances fare transparency, reduces booking delays, and ensures a structured ride management system. With a focus on digital transformation in urban mobility, NagarYatra provides an effective solution for both daily commuters and event-based travelers.

Keywords

Urban Mobility, Pre-booking Ride, Ride Booking System, Web-based Transport, PHP, MySQL, HTML, CSS, JavaScript, Real-Time Booking.

TABLE OF CONTENTS

<i>STUDENT DECLARATION</i>	ii
<i>CERTIFICATE FROM THE SUPERVISOR</i>	iii
<i>APPROVAL SHEET</i>	iv
<i>ACKNOWLEDGEMENT</i>	v
<i>ABSTRACT</i>	vi
<i>LIST OF FIGURES</i>	viii
<i>LIST OF TABLES</i>	ix
<i>LIST OF ABBREVIATIONS</i>	x
Chapter 1: Introduction	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objectives.....	1
1.4 Review of Related Work and Literature	1
1.4.1 Review of Related Work	1
1.4.2 Literature Review	2
1.5 Development Methodology.....	4
1.6 Scope and Limitation	5
1.6.1 Scope	5
1.6.2 Limitation	5
Chapter 2: System Development Process	7
2.1 Analysis.....	7
2.1.1 Requirement Analysis.....	7
2.1.2 Feasibility Analysis	7
2.1.3 Structured Modelling.....	8
2.2 Design.....	13
2.2.1 User Interface Design	13
2.2.2 Database Design Models	15
2.3 Implementation.....	23
2.3.1 Tools and Technologies Used	23
2.3.2 Module Description	25
2.3.3 Testing	25
Chapter 3: Conclusion and Recommendation.....	29
3.1 Summary	29
3.2 Conclusion.....	29
3.3 Future Recommendation	29
REFERENCES	a
APPENDICES	xi

LIST OF FIGURES

Figure No.	Title	Page Number
Figure 1.1:	Incremental Model	4
Figure 2.1:	Context Diagram of NagarYatra	8
Figure 2.2:	Level 1 DFD of NagarYatra.....	9
Figure 2.3:	Level 2 DFD of NagarYatra.....	10
Figure 2.4:	Gantt Chart of NagarYatra	11
Figure 2.5:	Signup UI of NagarYatra	13
Figure 2.6:	Login Interface of NagarYatra	13
Figure 2.7:	User Dashboard of NagarYatra	14
Figure 2.8:	Admin Dashboard of NagarYatra.....	14
Figure 2.9:	ER Diagram of NagarYatra.....	15
Figure 2.10:	Database Schema Design of NagarYatra	16
Figure 2.11:	Architectural Design of NagarYatra.....	17
Figure 2.12:	Use Case of NagarYatra	18

LIST OF TABLES

Table No.	Title	Page Number
Table 2.1:	Use Case Description (Login)	19
Table 2.2:	Use Case Description (User Registration).....	19
Table 2.3:	Use Case Description (Add Vehicle).....	19
Table 2.4:	Use Case Description (Change Vehicle)	20
Table 2.5:	Use Case Description (Approve/Reject vehicle)	20
Table 2.6:	Use Case Description (Booking Request)	20
Table 2.7:	Use Case Description (Get OTP).....	20
Table 2.8:	Use Case Description (View OTP).....	21
Table 2.9:	Use Case Description (View Ride Status).....	21
Table 2.10:	Use Case Description (Accept/Reject Ride).....	21
Table 2.11:	Use Case Description (View Report)	22
Table 2.12:	Use Case Description (Submit Feedback)	22
Table 2.13:	Use Case Description (View Feedback).....	22
Table 2.14:	Use Case Description (Logout)	23
Table 2.15:	Test Case 001 - Registration form	25
Table 2.16:	Test Case 002 - Login form	25
Table 2.17:	Test Case 003 – Algorithm	26
Table 2.18:	Test Case 004 – Booking Form	26
Table 2.19:	Test Case 005 – Booking Request Status	26
Table 2.20:	Test Case 006 – Reset Password (Forgot Password).....	27
Table 2.21:	Test Case 007 – Change Password	27
Table 2.22:	Test Scenario 00_01- Proceeding for Booking.....	27

LIST OF ABBREVIATIONS

Abbreviations	Full Forms
AJAX	Asynchronous JavaScript and XML
CSS	Cascading Stylesheets
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
HTML	Hypertext Markup Language
JS	JavaScript
PDF	Portable Document Format
PHP	Hypertext Pre-processor
TC	Test Case
UI	User Interface
UC	Use Case
VS code	Visual Studio code
WYSIWYG	What You See Is What You Get

Chapter 1: Introduction

1.1 Background of the Project

NagarYatra a web application for booking the E-Safari, Bike, Auto, etc. and for booking them for long period i.e., 4-5 days, or for pre-booking for a particular event. Transportation within cities has always been a challenge, especially in terms of time management and cost transparency. NagarYatra aims to address these issues through a web-based application that enables users to book city transport services quickly and efficiently. This system ensures a secure, transparent, and user-friendly experience for citizens, drivers, and administrators alike.

1.2 Problem Statement

Urban transportation faces several challenges, including inefficiency, lack of cost transparency, and frequent time delays, making it inconvenient for citizens. Existing traditional systems of booking often fail to provide a seamless and secure booking experience, leaving users uncertain about ride availability, pricing, and service reliability. Additionally, drivers encounter difficulties in managing rides due to the absence of streamlined tools, leading to operational inefficiencies. Administrators also struggle with effective oversight and monitoring of city transport services, resulting in disorganized operations. To address these issues, there is a need for a centralized, web-based platform that simplifies the booking process for users, ensures transparency in costs and services, and improves ride management efficiency for drivers. Implementing such a solution will enhance the overall transportation experience, improve efficiency, and build trust among all stakeholders, making urban mobility more accessible and reliable.

1.3 Objectives

The objective of the NagarYatra is as follows:

- ✓ To transform the traditional booking process into an efficient, digital system.
- ✓ To provide transparency in pricing, ensuring passengers are charged fairly.
- ✓ To enable users to locate and book the nearest vehicles in real-time.

1.4 Review of Related Work and Literature

1.4.1 Review of Related Work

In the field of urban transportation, booking reliable and affordable city transport services has become a crucial necessity. Whether it is for daily commutes(journey), long-term booking, or special event travel, users seek a seamless and transparent booking system. NagarYatra is designed to meet these needs by providing a web-based platform that enables users to book E-Safari, bikes, autos, and other transport services efficiently. The core concept of NagarYatra revolves around enhancing accessibility,

cost transparency, and service reliability while ensuring a smooth booking experience for passenger and driver.

The development of NagarYatra involves the study of existing systems such as "Pathao", "InDriver", "TaxiMandu", "Sajilo Rider" and "kawa Rides", where users can book rides based on location and fare estimates. These platforms provide basic functionalities such as ride requests, fare calculations, and driver details. However, NagarYatra enhances this approach by offering a more structured system that allows users to pre-book vehicles for a longer duration, access real-time booking, and ensure secure transactions, making it a more versatile and efficient solution.

One of the key conclusions drawn from the study of existing systems is that instead of providing direct driver details to users, it is more effective to implement a request-based system where users send booking requests and drivers accept them before sharing their information. This enhances security and prevents misuse of driver data. Additionally, NagarYatra integrates a location-based service to find the nearest available by using the longitude and latitude, ensuring convenience and efficiency in urban transportation.

1.4.2 Literature Review

Pathao (Pathao, n.d.) is a web and mobile application that facilitates ride-hailing services and package delivery through cloud-based communication. The platform allows passengers to find nearby drivers through a mobile application, displaying a list of available drivers with ratings from previous passengers. Specifically, this study aims to develop an efficient mobile application for ride-hailing services that provides passengers with essential driver information, ensuring they can select an appropriate driver for their travel needs, integrate GPS tracking to monitor ride locations in real time, and evaluate the system's usability and functionality. Traditional ride-hailing services in urban areas often face issues related to availability, pricing transparency, and security. The Pathao system was designed to streamline ride bookings by providing real-time driver tracking, fare estimates, and user reviews. Additionally, a security feature ensures that ride details, including driver identity and vehicle information, are accessible to the user before booking confirmation. The system enables a more convenient and safer ride-hailing experience, mitigating concerns over fraudulent drivers and unsafe ride locations. The usability and functionality evaluation suggests that passengers benefit from quick ride-hailing, transparent pricing, and real-time monitoring, ensuring a seamless travel experience.

InDriver (InDriver, n.d.) is an innovative mobile application that introduces a unique real-time bargaining feature for ride fares, giving passengers the ability to negotiate pricing directly with drivers. The system connects passengers and drivers via cloud-based communication, ensuring availability through real-time location tracking. Specifically, the study aims to develop a mobile application that enhances the ride-hailing process by allowing passengers to set a proposed fare, integrate a GPS-based

system to track driver and ride locations, and evaluate the effectiveness of the platform in terms of usability and functionality. Unlike traditional ride-hailing apps, which use fixed pricing algorithms, InDriver's unique feature allows passengers to propose a fare, and nearby drivers can accept, decline, or negotiate. This decentralized pricing model provides flexibility and promotes fairer pricing based on demand. Security concerns regarding passenger safety are mitigated through the GPS tracking system, which shares ride details with emergency contacts if necessary. Based on usability and functionality evaluations, the study concludes that InDriver provides a dynamic and passenger-friendly alternative to traditional ride-hailing services, allowing for cost-effective and secure transportation options.

TaxiMandu (TaxiMandu, n.d.) is a Nepal-based mobile application designed for efficient taxi booking with enhanced security and transparency. The system facilitates ride-hailing through a cloud-based platform where passengers can book taxis with predefined fare structures and track ride progress via GPS. Specifically, the study aims to develop a mobile application that simplifies taxi booking with fare transparency, integrate GPS tracking to enhance passenger safety, and assess the system's usability and functionality. Unlike informal taxi services where fares may be negotiated manually, TaxiMandu provides standardized pricing, reducing fare disputes and ensuring fair charges for passengers. Additionally, the system issues a digital ride receipt that includes the driver's details, estimated fare, and ride duration. A unique security feature enables passengers to share their real-time location with trusted contacts. Evaluation results indicate that TaxiMandu enhances passenger safety and convenience by combining structured pricing with real-time tracking, making urban transportation more reliable and efficient.

Sajilo Rider (Sajilo Rider, n.d.) is a Nepal-based ride-hailing service designed to offer affordable and accessible bike and car transportation primarily within the Kathmandu Valley. The app allows passengers to book nearby rides using GPS-based technology and real-time driver availability. The key objectives of the Sajilo Rider platform are to create a simple and efficient interface for ride booking, provide affordable and transparent pricing for daily commuters, and ensure safe and trusted rides by verifying both riders and drivers. Sajilo Rider emphasizes a user-friendly design and incorporates features like estimated fare calculation, ride history, and contactless payment options. The platform is aimed at improving the everyday commuting experience by addressing gaps in public transport systems and offering a domestic alternative to international ride-hailing apps. By promoting local employment through its rider network, Sajilo Rider also contributes to Nepal's digital service economy. The service's streamlined approach to booking and its adaptability to Nepalese traffic conditions make it a valuable addition to the growing ride-hailing ecosystem in the country.

Kawa Rides (Kawa Rides, n.d.) is a locally developed ride-hailing service in Nepal offering both bike and car rides through a user-friendly mobile application. Positioned as an alternative to popular platforms like Pathao and Tootle, Kawa Rides emphasizes

ease of access, fair pricing, and localized service. The platform's core objectives are to provide an affordable and reliable commuting option for urban users, allow users to book rides via app-based GPS services, and increase market competition by promoting Nepal-made technological solutions. Kawa Rides also enables riders to register and earn income, thus supporting local employment. In comparison to its competitors, Kawa Rides attempts to offer a more community-centric experience, tailoring features to fit Nepalese commuting habits and urban road conditions. With its focus on both bike and car rides, Kawa Rides seeks to bridge the gap between economic two-wheeler transport and more comfortable car options for everyday travel.

These ride-hailing platforms-Pathao, InDriver, TaxiMandu, Sajilo Rider and Kawa Rides.

Technological advancements in urban mobility, integrating location tracking and security measures to improve user experience. The studies conclude that driver rating systems, and fare transparency contribute significantly to passenger trust and satisfaction in ride-hailing services.

1.5 Development Methodology

The incremental model divides the NagarYatra development process into smaller, manageable increments, allowing for gradual implementation and refinement. Each increment delivers a functional component of the system, enabling early user feedback and adjustments. This approach enhances flexibility in accommodating evolving requirements while maintaining a structured development process.

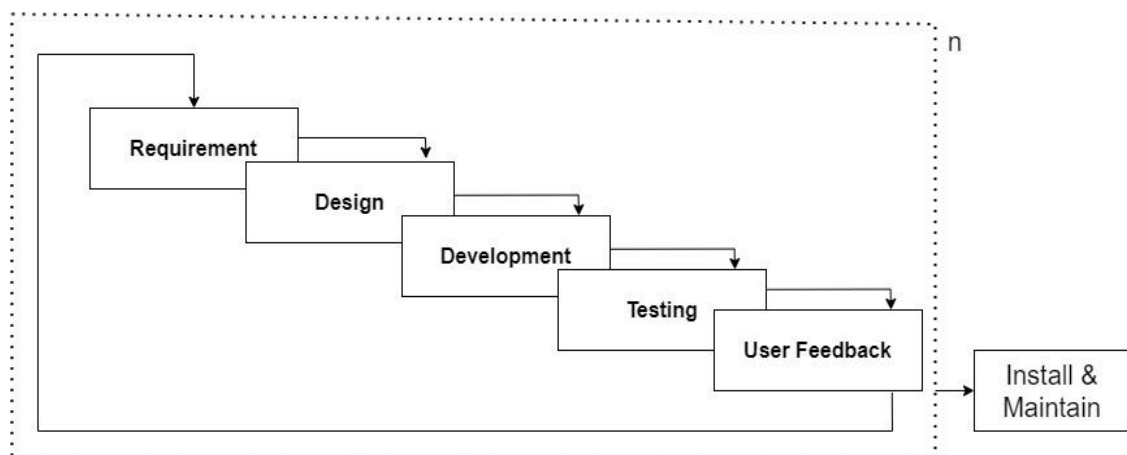


Figure 1.1: Incremental Model

Requirement gathering:

In this phase, various requirements such as user roles, booking preferences, and transport service types in the NagarYatra application are identified. The analysis includes essential features like real-time vehicle availability, fare estimation, and location-based service discovery. Additionally, system requirements for ensuring

secure transactions, user authentication, and seamless booking processes are gathered to enhance the overall efficiency and usability of the platform.

Design:

Here, based on the requirement gathering, user interface design including layout, colors, typography, arrangement of components in web application is done. Responsive design considering different screen sizes and device orientation is done to ensure the website is responsive.

Development:

In this phase, coding of NagarYatra starts following the website architecture and design guidelines. Small chunks of program are written, and testing is also done.

Testing:

In testing phase, identification of types of tests to be conducted such as functional, usability, performance, and security testing of NagarYatra. Test cases are created based on our website requirement and are executed where input for the test is taken, interaction with the website and comparison of the actual results with the expected result is done. After overall system testing is done, the product is deployed into the market.

Maintenance:

In the maintenance phase, NagarYatra will ensure that website remains stable, performs well, addresses user concerns, and continues to evolve to meet the changing needs of the users.

1.6 Scope and Limitation

1.6.1 Scope

The NagarYatra web application is a digital platform designed to streamline city transportation by enabling users to book E-Safari, bikes, autos, and other transport services for daily commutes, long-term booking, and special events. It addresses key challenges such as inefficiency, unclear pricing, and ride availability by offering real-time booking, transparent fare management, and location-based vehicle discovery. With a secure and user-friendly interface for citizens, drivers, and administrators, it enhances operational efficiency and oversight. By integrating near-by active drivers and automated estimated pricing, NagarYatra aims to improve service reliability, accessibility, and organization, making city transport more efficient and trustworthy.

1.6.2 Limitation

It does not offer real time tracking and doesn't support multiple vehicles of a single driver.

1.7 Report Organization

The report consists of three chapters which will cover the designing and development application.

Chapter One introduces the system and the problems and literature review which is the previous related work that has been done before and gives an overview of the study. Similarly, Chapter Two cover the analysis, designing and implementation and testing. And Chapter Three discusses the summary, conclusion, recommendations for future works to improve this study.

Chapter 2: System Development Process

2.1 Analysis

In this phase plans are made regarding to the developing process of the NagarYatra. The study of functional and non-functional requirements is done.

2.1.1 Requirement Analysis

Here, the gathering of the requirements that are essential for the development of the website are done and analyzing its various parameters based on feasibility study and so on.

i. Functional Requirements

It defines functioning or behavior of the system.

- i. Register/Login to the Website with valid email and password.
- ii. User can select Booking i.e., Pre-booking or real time booking.
- iii. User can proceed for Booking.
- iv. User and driver will be notified about their Booking request status through email.

ii. Non-Functional Requirements

It defines the criteria according to which the system must work.

Availability:

The website is made available 24/7 that handles and supports multiple users at a time and quick response time.

Security:

Users will have to authenticate using the email and password. Depending on the access level user's authorization is given.

Usability:

The interfaces designed is kept simple with user friendly interface that makes it easy to search for and find suitable Vehicle as per their requirements.

2.1.2 Feasibility Analysis

A feasibility analysis of NagarYatra involves various aspects of the website design, development, and implementation to determine whether it is likely to be successful. Here are some factors:

i. Technical Feasibility

NagarYatra is a responsive web app optimized for various devices. The main technologies associated with this project are: Front end: HTML, CSS, JS. Back-end: PHP, MySQL. Each of the technologies are freely available and the technical skills required are manageable. So, it is technically feasible. This system requires the less technical skills to operate it.

ii. Economical Feasibility

The developer doesn't need to pay for the tools and technologies used to develop the system. Additionally, the Users (Drivers and Passenger) don't need to pay to use this system. So, it is economical feasible.

iii. Operational Feasibility

This system will provide consistent and integrated data management. It reduces the effort of Users (Passenger and Driver) and will be simple and easy to use making the system operationally feasible.

2.1.3 Structured Modelling

i. Context Diagram

The context diagram of NagarYatra provides a high-level overview of the system's interaction with external entities. It outlines how data flows between the system and external users or components without detailing the internal processes.

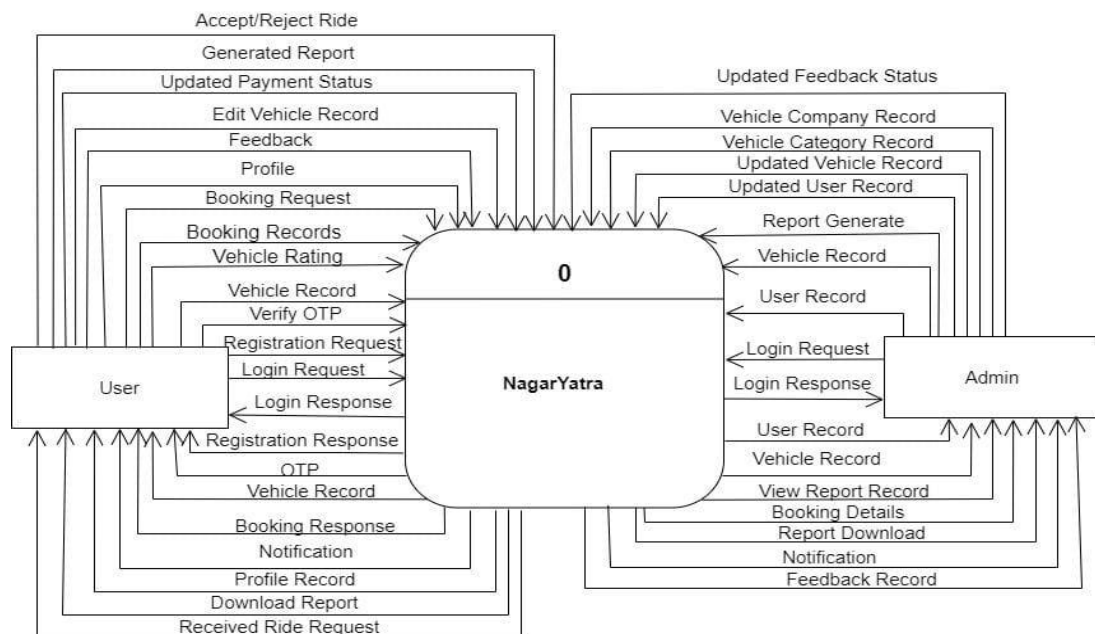


Figure 2.1: Context Diagram of NagarYatra

In the context of NagarYatra, the main external entities include User (Passengers and Drivers), and Admin. Passengers can register, login, request rides by providing ride details, view ride status, and receive notifications. Drivers can accept or reject ride requests, update ride status, and view user details. The admin manages user accounts, monitors bookings, verifies vehicles, and oversees system operations.

ii. Level 1 DFD

The Level 1 DFD of NagarYatra breaks down the system into main functional processes such as user registration/login, notifications handling and many more.

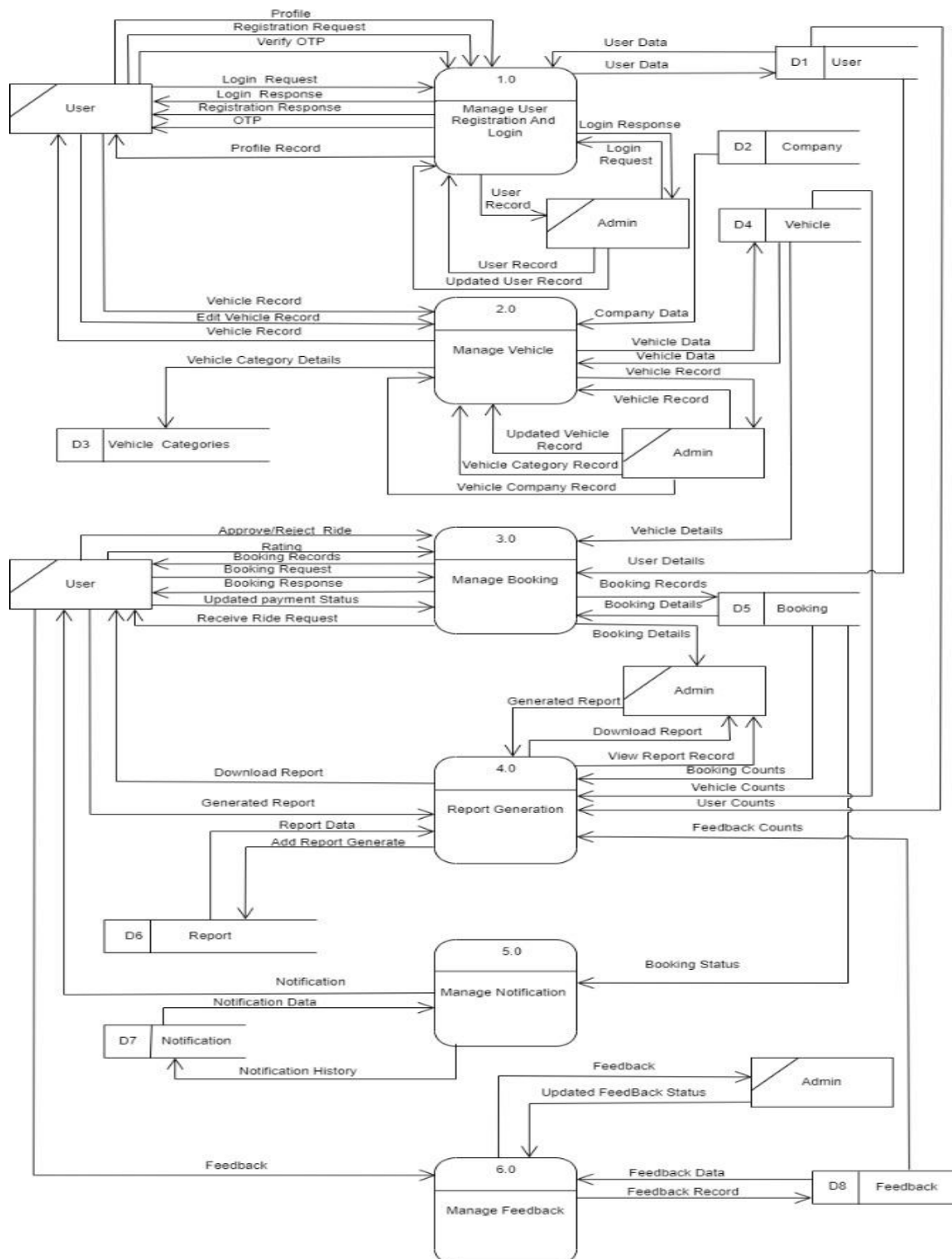


Figure 2.2: Level 1 DFD of NagarYatra

In Level 1 DFD, when a user books a ride, the system stores the request, calculates distance and fare, and sends the request to nearby drivers. Drivers can then accept or reject the ride.

iii. Level 2 DFD

The Level 2 DFD of NagarYatra provides a more detailed view of the internal processes involved in each major function shown in the Level 1 DFD.

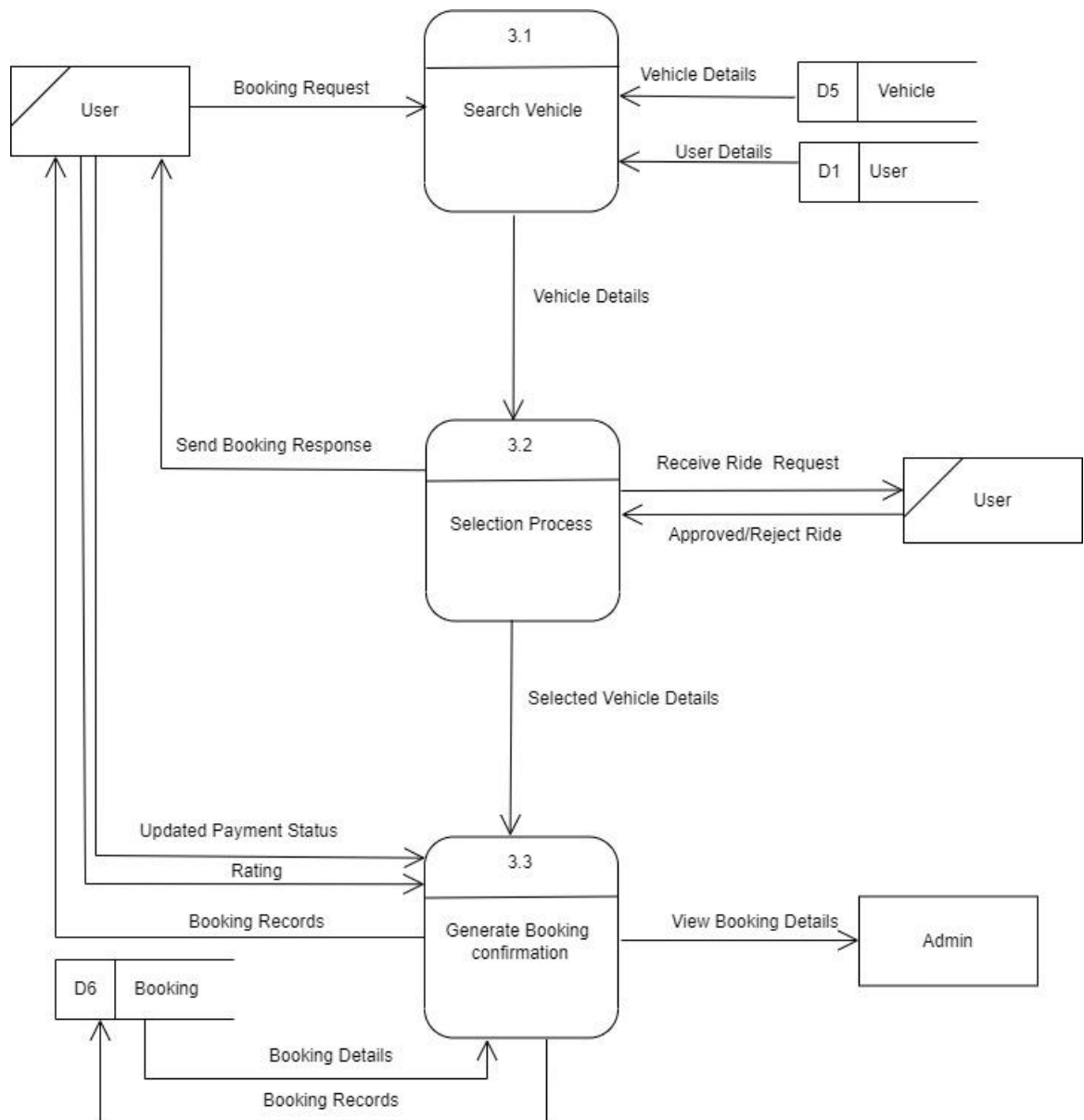


Figure 2.3: Level 2 DFD of NagarYatra

In Level 2 DFD, we have further decomposed Booking management process into specific sub-processes such as Search Ride, Selection Process which is automatically performed by the system based on the location of the nearest User(driver) and Generate booking confirmation and Users receives the notifications.

iv. Time Analysis

Here is the Gantt chart showing the probability of a project to be completed within its schedule time limits by a planned week.

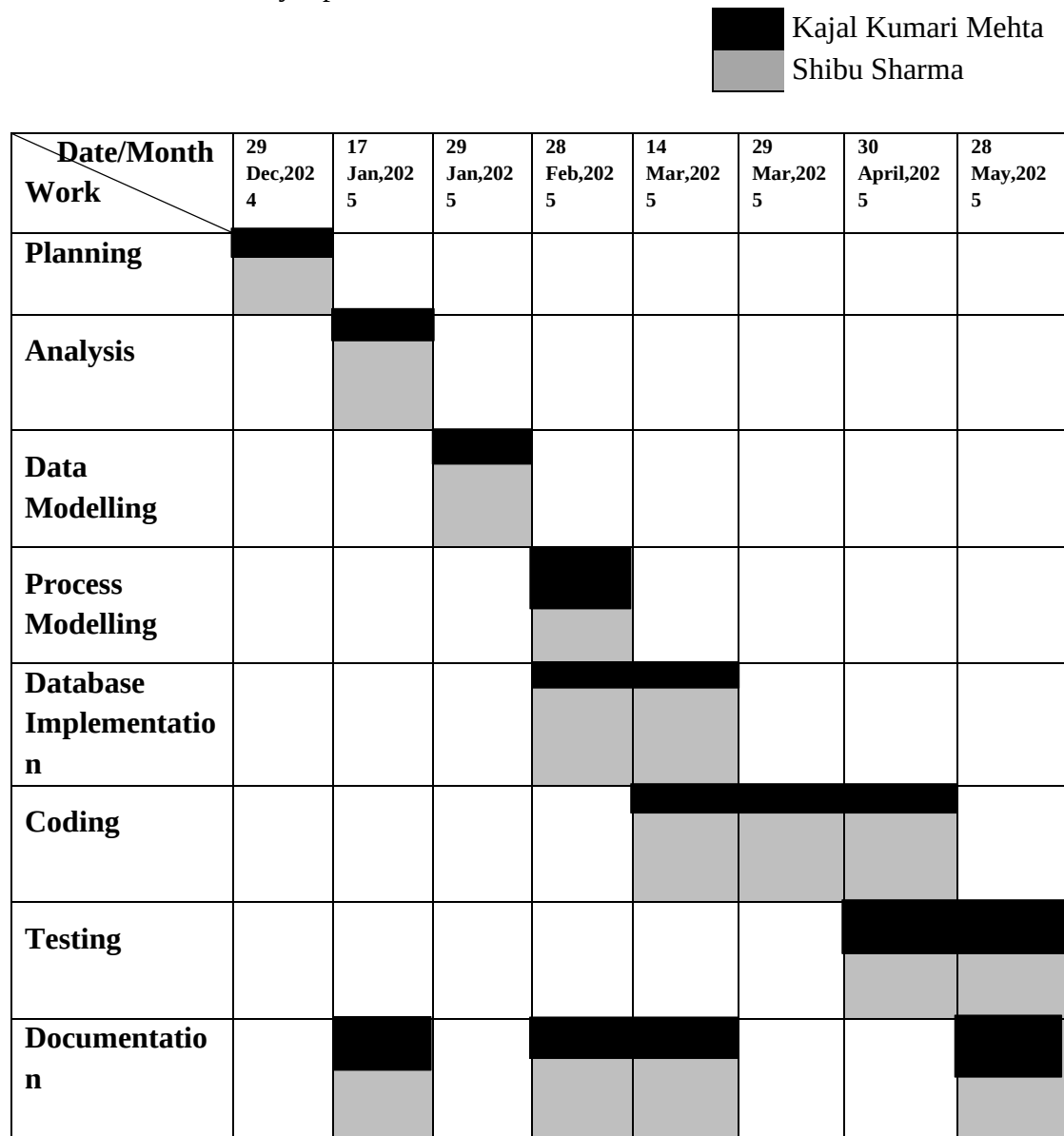


Figure 2.4: Gantt Chart of NagarYatra

v. Algorithm details

Haversine formula

Haversine formula (Nichat & Gadicha, 2015) determines the great circle distance (i.e., the shortest distance) between two points on sphere, such as the earth. It is based on the law of haversines, which relates the sides and angles of a spherical triangle. To calculate the great circle distance between two points it uses the central angle between the two points.

We have used the haversine formula to calculate the distance between the passenger and driver and send the booking request to the driver who is in the radius of 10 km from the passenger.

Code snippet:

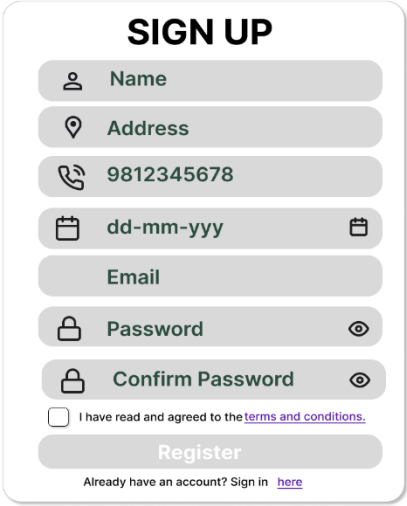
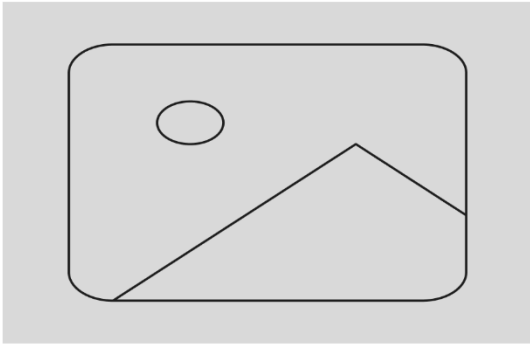
```
def calculate_distance (lat1, lon1, lat2, lon2):  
    R = 6371.0  
    dlat = radians(lat2-lat1)  
    dlon = radians(lon2-lon1)  
    a = sin(dlat/2) **2+cos(radians(lat1)) *cos(radians(lat2)) *sin(dlon/2) **2  
    c = 2*atan2(sqrt(a), sqrt(1-a))  
    distance = R*c  
    return distance.
```

2.2 Design

2.2.1 User Interface Design

i. Registration Interface

The registration page of NagarYatra allows new users to create an account by entering details like name, email, phone number, and password. It validates the input, encrypts the password, and stores the information securely.



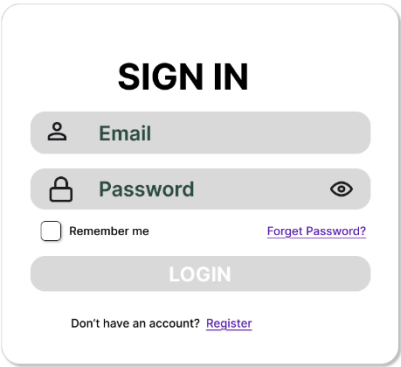
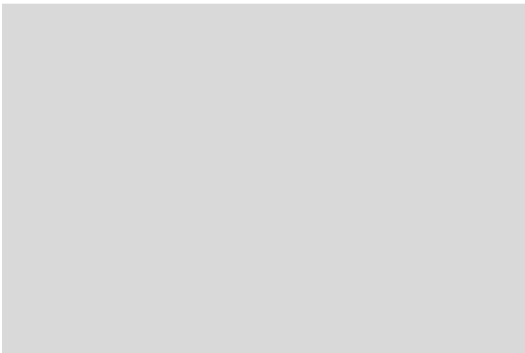
The registration form, titled "SIGN UP", contains the following fields and elements:

- Name
- Address
- Phone number: 9812345678
- Date of birth: dd-mm-yyy
- Email
- Password (with an eye icon for visibility toggle)
- Confirm Password (with an eye icon for visibility toggle)
- A checkbox for "I have read and agreed to the [terms and conditions](#)."
- A "Register" button.
- A link at the bottom: "Already have an account? Sign in [here](#)".

Figure 2.5: Signup UI of NagarYatra

ii. Login Interface

The login page of NagarYatra allows registered users to access their accounts by entering their email or phone number and password. It verifies the credentials against stored records and grants access if the information is correct. If the login fails, it shows an error message. This page is essential for users to securely access features like booking rides, viewing trip history, or managing their profiles.



The login form, titled "SIGN IN", contains the following fields and elements:

- Email
- Password (with an eye icon for visibility toggle)
- A checkbox for "Remember me" and a link for "Forget Password?".
- A "LOGIN" button.
- A link at the bottom: "Don't have an account? [Register](#)".

Figure 2.6: Login Interface of NagarYatra

iii. User Dashboard Interface

The user dashboard of NagarYatra is the main interface users see after logging in. It provides quick access to key features like booking a ride, viewing current and past bookings, checking ride status, and managing their profile. The dashboard is designed to be simple, responsive, and user-friendly, allowing users to navigate the platform easily and make ride requests efficiently.

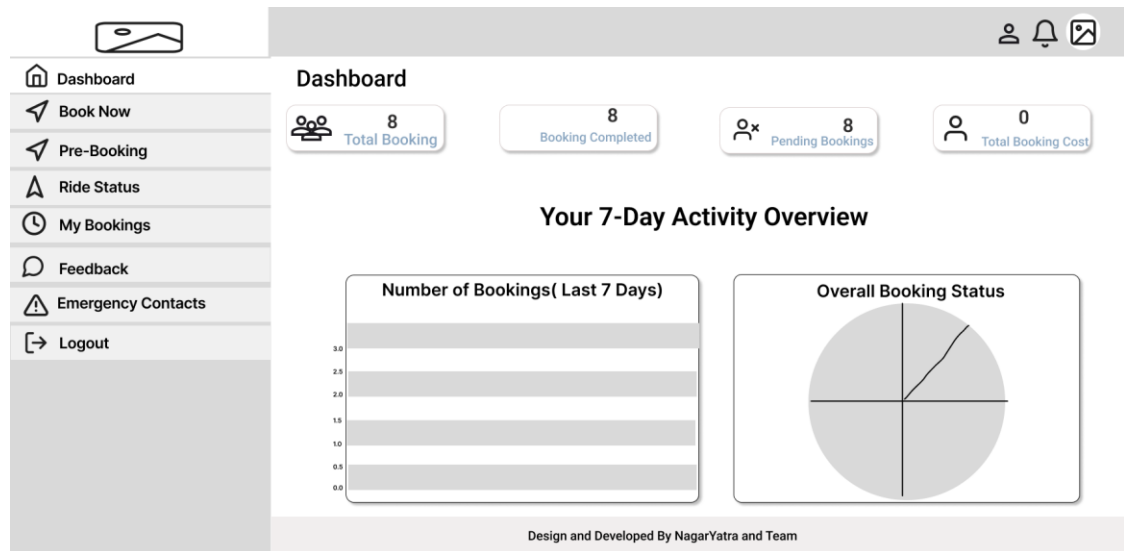


Figure 2.7: User Dashboard of NagarYatra

v. Admin Dashboard

The admin dashboard of NagarYatra is a control panel for managing the entire platform. It allows the admin to monitor and manage users, drivers, vehicle listings, bookings, and ratings. The dashboard provides an overview of system activity, such as total bookings, active users, pending approvals, and more. It also includes tools for generating reports, and maintaining system settings. The admin dashboard is designed to be secure, organized, and easy to use for efficient platform management.

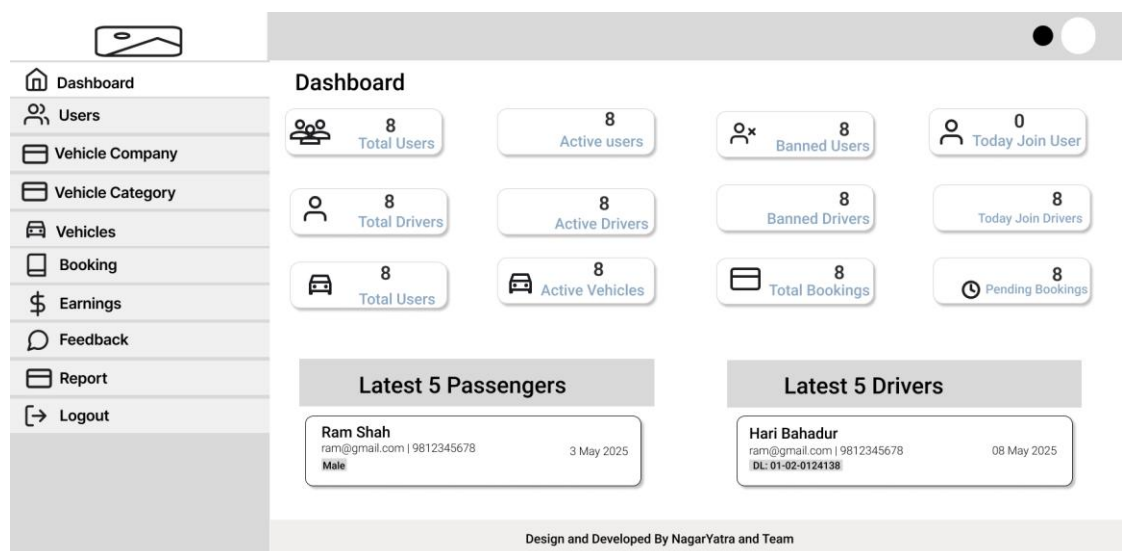


Figure 2.8: Admin Dashboard of NagarYatra

2.2.2 Database Design Models

i. Conceptual Diagram

The conceptual diagram of NagarYatra represents the high-level structure of the system by identifying its main entities and the relationships among them. Key entities of NagarYatra include User, Booking, Vehicle, Vehicle Category, Vehicle Company, Feedback, and Notification.

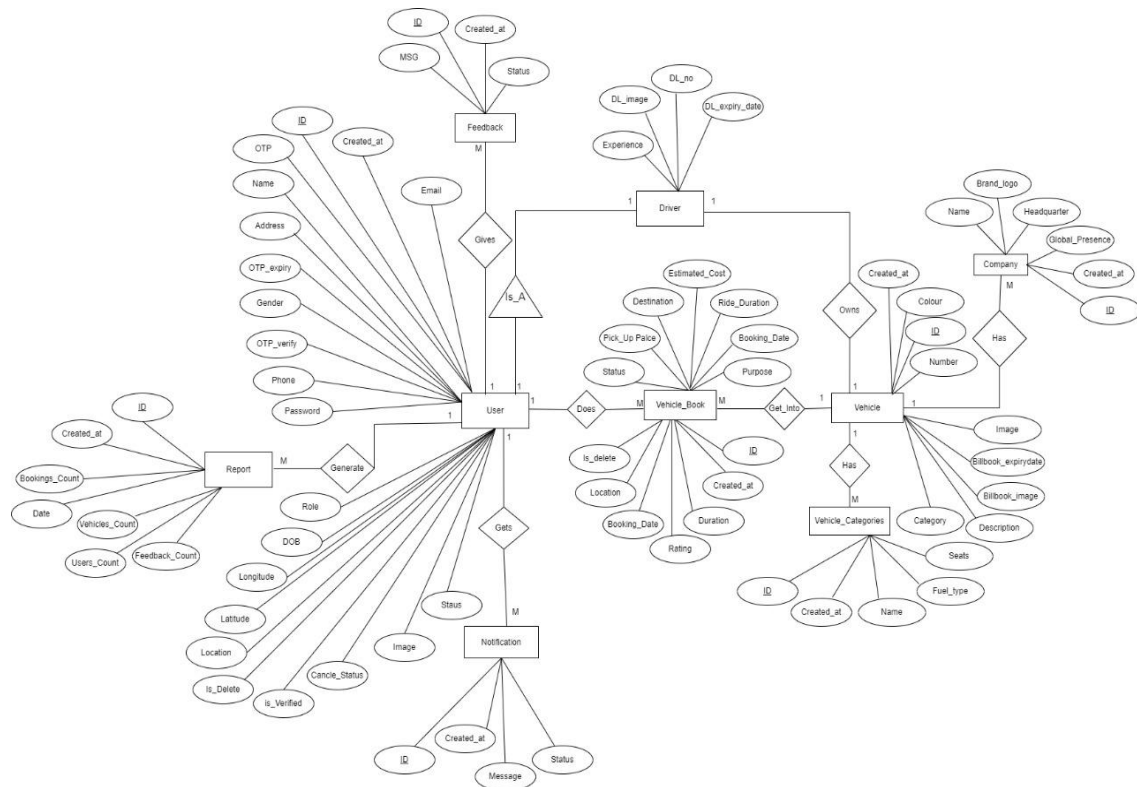


Figure 2.9: ER Diagram of NagarYatra

Relations

Admin (id (PK), ...)

User (id (PK), ...)

Vehicle_Book (id (PK), user_id (FK), ...)

Vehicle_category (id (PK),..)

Vehicle_Company (id (PK),)

Vehicle (id (PK),...)

Feedback (id (PK),)

Notification (id (PK))

ii. Physical Diagram

The physical schema of NagarYatra defines how data is actually stored in the database, including the tables, fields, data types, relationships, and constraints. It translates the conceptual design of NagarYatra into a real database structure using MySQL. It clearly defines primary keys and foreign key relationships to ensure data integrity.

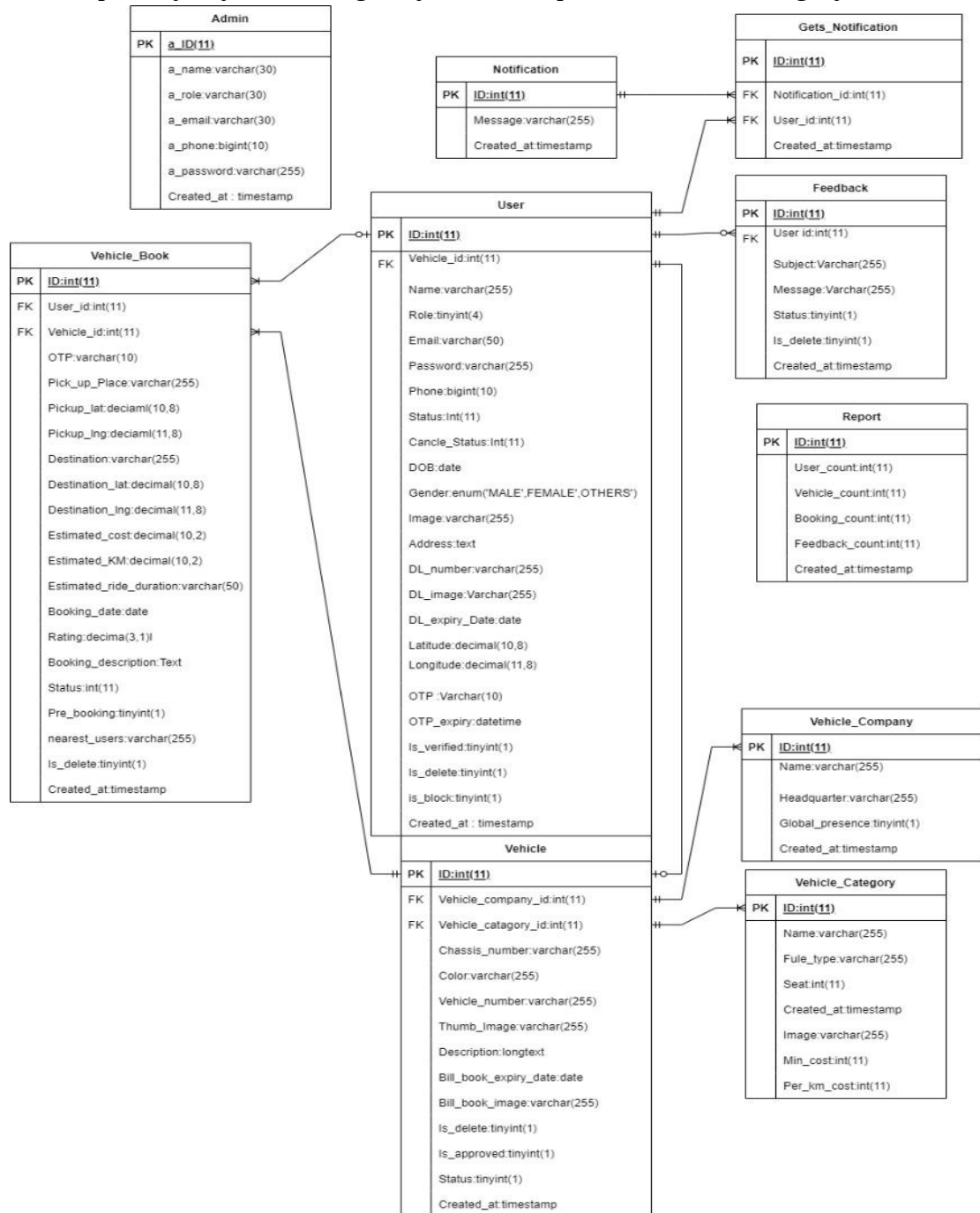


Figure 2.80: Database Schema Design of NagarYatra

iii. Architectural Design

The architectural design of NagarYatra is based on a three-tier architecture, which includes the Presentation Layer, Application Layer, and Data Layer.

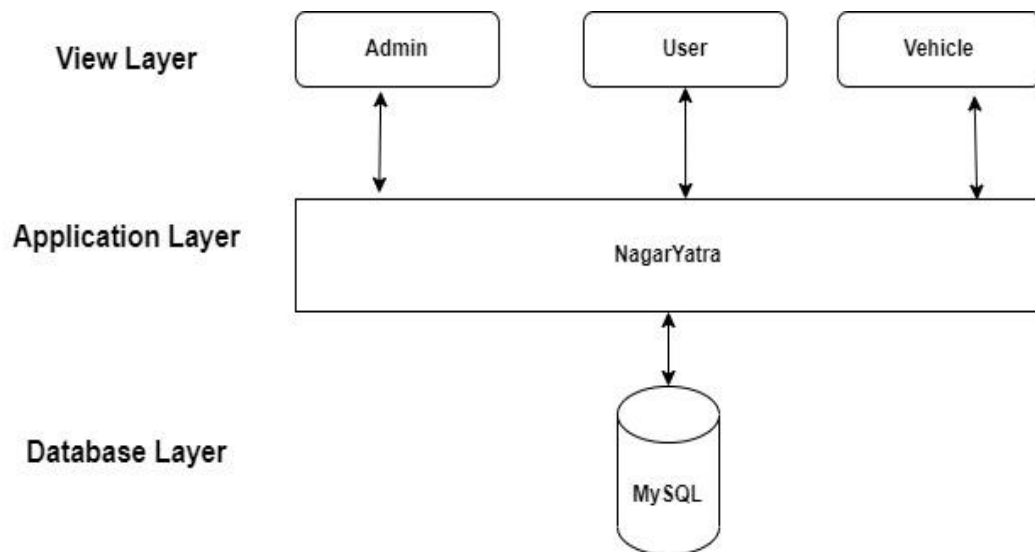


Figure 2.91: Architectural Design of NagarYatra

“NagarYatra” follows 3 tier architecture where each layer is independent of one another. The View layer has two entities i.e., Admin and User (Passenger and Driver). Application layer is NagarYatra itself and the third layer is Database layer which is MySQL.

iv. Use Case Diagram

A use case of NagarYatra represents a specific interaction between a user (passenger, driver), Admin and the system to achieve a particular goal, such as booking a ride, accepting a request, or managing users.

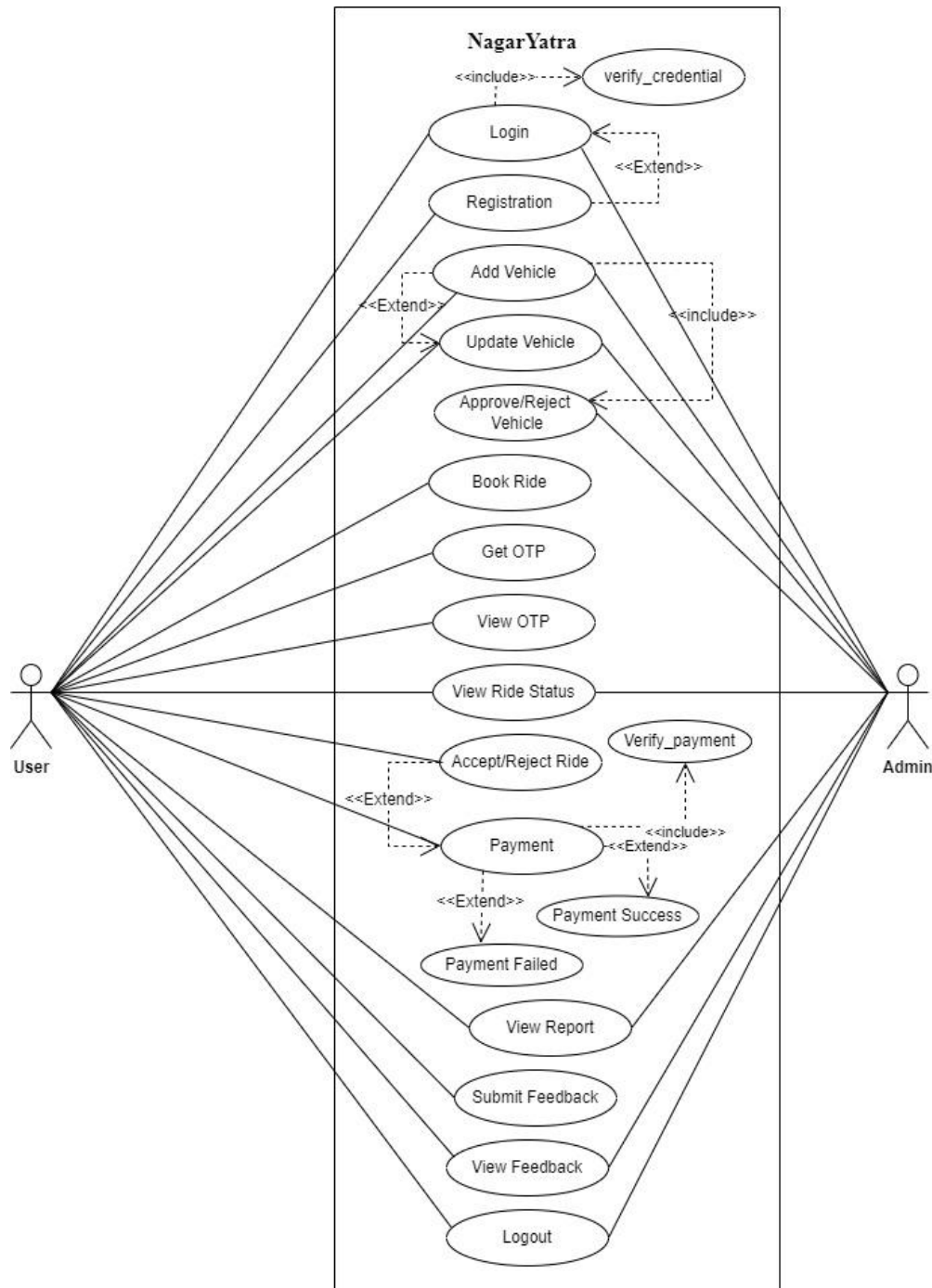


Figure 2.12: Use Case of NagarYatra

Use Case Description Tables

Table 2.1: Use Case Description (Login)

Use Case ID	UC-001
Primary Actor	Admin, Users (Passenger/Driver)
Secondary Actor	None
Description	The user log in using valid credentials.
Preconditions	Users must be registered and verified
Postconditions	Users can access to the system.
Success Scenario	Users enters the valid login credentials, then the system verifies it, and the user successfully login.
Failure Scenario	Login unsuccessful if credentials are incorrect, or system error occurs.

Table 2.2: Use Case Description (User Registration)

Use Case ID	UC-002
Primary Actor	Users (Passenger/Driver)
Secondary Actor	None
Description	The user registers an account using an valid email and other details.
Preconditions	User must have a valid email and accept terms and conditions.
Postconditions	User account is created then email is sent confirming successful registration and User redirected to login page.
Success Scenario	User enters details, system validates, sends OTP, user enter valid OTP, and account is created.
Failure Scenario	Invalid email format, OTP not received or entered incorrectly, or email already registered.

Table 2.3: Use Case Description (Add Vehicle)

Use Case ID	UC-003
Primary Actor	User (Driver)
Secondary Actor	Admin
Description	User adds a Vehicle to the system with proper validated details and later on admin verifies it.
Preconditions	User must be logged in and has driver account.
Postconditions	Vehicle is successfully added to the system list.
Success Scenario	User enters vehicle details, system validates input, and Vehicle is added successfully.
Failure Scenario	If credentials are incorrect, account is unverified, or system error occurs.

Table 2.4: Use Case Description (Change Vehicle)

Use Case ID	UC-004
Primary Actor	User (Driver)
Secondary Actor	Admin
Description	User updates an existing vehicle with new vehicle.
Preconditions	Users must be logged in and have an existing vehicle.
Postconditions	Vehicle is Changed with add details.
Success Scenario	Users modify details of existing, and system save changes and Admin approves it.
Failure Scenario	Admin rejects the changes as s/he finds the details of vehicle is invalid.

Table 2.5: Use Case Description (Approve/Reject vehicle)

Use Case ID	UC-005
Primary Actor	Admin
Secondary Actor	User (Driver)
Description	Users adds a vehicle and later on admin approve/reject the vehicle on the basis of details provide by user.
Preconditions	Admin must be logged in and users have an added/changed the vehicle.
Postconditions	Vehicle is approved/rejected from the system by the admin.
Success Scenario	User adds/change the vehicle and Admin approves it.
Failure Scenario	User adds/change the vehicle and Admin rejects it as the details was found invalid.

Table 2.6: Use Case Description (Booking Request)

Use Case Identifier	UC-006
Primary Actor	Passenger
Secondary Actor	Driver
Description	The ride is booked by the Passenger from the specific place to the specific destination.
Pre-Condition	Vehicle must be available near by the passenger or which is selected by the passenger.
Post-Condition	Ride has been accepted by the driver and the OTP has been sent to the user for the further verification.
Failure-Scenario	Vehicle couldn't be booked by the Passenger as it is unavailable.

Table 2.7: Use Case Description (Get OTP)

Use Case ID	UC-007
-------------	--------

Primary Actor	User (Passenger/Driver)
Secondary Actor	None
Description	Users gets the OTP when s/he books a ride.
Preconditions	User must be logged in and user has booked a ride or is in registration process.
Postconditions	User gets the OTP and enters it, and system validates it.
Success Scenario	User enters project details; system validates is as correct.
Failure Scenario	Fails if the entered OTP is invalid i.e., it doesn't match with the OTP of the database, or a system error occurs.

Table 2.8: Use Case Description (View OTP)

Use Case ID	UC-008
Primary Actor	User (Passenger/Driver)
Secondary Actor	None
Description	User can view the OTP in Email when registering and in the Email and Ride Status after booking a ride.
Preconditions	Users must have submitted the registration form or booked the ride.
Postconditions	User can view the OTP and continue the further process of register or booking.
Success Scenario	User gets the OTP and can view it in the email or in dashboard.
Failure Scenario	User didn't get the OPT due to incorrect email address or a system error occurs.

Table 2.9: Use Case Description (View Ride Status)

Use Case ID	UC-009
Primary Actor	User (Passenger/Driver)
Secondary Actor	None
Description	User can view the Ride Status after booking a ride.
Preconditions	Users must be logged in and booked the ride.
Postconditions	User can view the Ride Status.
Success Scenario	User have booked a ride and it can be seen in the seen in the Dashboard.
Failure Scenario	User is not logged in or a system error occurs.

Table 2.10: Use Case Description (Accept/Reject Ride)

Use Case ID	UC-010
Primary Actor	User (Driver)
Secondary Actor	User (Passenger)

Description	User (Passenger) books a ride and later on Users (Driver) accept/reject the ride.
Preconditions	User (Passenger) must be logged in and booked a ride.
Postconditions	User (Driver) accept/reject from the ride as per his/her convenience.
Success Scenario	User (Passenger) books the ride and User (Driver) accept the ride.
Failure Scenario	User (Driver) rejects the ride as he was unavailable.

Table 2.11: Use Case Description (View Report)

Use Case ID	UC-011
Primary Actor	Admin or User (Driver)
Secondary Actor	None
Description	Admin or User (Driver) can view the Monthly Report and can download it in PDF format.
Preconditions	Admin or User (Driver) must be logged in.
Postconditions	Admin or User (Driver) can view and download the PDF.
Success Scenario	Admin or User (Driver) can view the Report and Download it.
Failure Scenario	User is not logged in or no data is found to generate report, or a system error occurs.

Table 2.12: Use Case Description (Submit Feedback)

Use Case ID	UC-012
Primary Actor	User
Secondary Actor	Admin
Description	User submits feedback along with the subject.
Preconditions	User must be logged in.
Postconditions	Feedback is recorded and Admin can view it and contact him/her in case of issue (if any).
Success Scenario	User gives the feedback; system saves and displays it to Admin.
Failure Scenario	Fails if the input field as left blank or a system error occurs.

Table 2.13: Use Case Description (View Feedback)

Use Case ID	UC-013
Primary Actor	Admin
Secondary Actor	User
Description	Admin views the feedback sent by the user.
Preconditions	Feedback must exist and Admin is logged in.

Postconditions	Feedbacks are displayed along with the subject and email address of the user.
Success Scenario	Admin navigates to Feedback section; system retrieves and displays Feedback.
Failure Scenario	Fails if no reviews are available or a system error occurs.

Table 2.14: Use Case Description (Logout)

Use Case ID	UC-014
Primary Actor	Admin, Users
Secondary Actor	None
Description	Admin and users log out of the system and redirected index page.
Preconditions	Admin and User must be logged in.
Postconditions	Admin and user are logged out and redirected to the login screen.
Success Scenario	When User clicks logout, system ends the session, and user is redirected to login page.
Failure Scenario	System error preventing logout.

2.3 Implementation

In this phase working of the overall system is checked. It deals with the complete process of building and implementing it. It focuses on the technical aspects of the system starting with identifying the necessary components and building the relevant relationship among subsystems for the smooth and efficient operation of the system.

2.3.1 Tools and Technologies Used

i. Diagramming Tool

Draw.io: It is proprietary software for making diagrams and charts. We have used it to draw the ER diagram, Physical Schema, Architectural design, Use Case Diagram, Context diagram, Level 1 DFD and Level 2 DFD.

Figma: Figma is an online graphic design tool used for creating visually appealing designs with ease. We used Figma to design the user interface layouts of NagarYatra, including registration forms, login screens, dashboards, and booking pages, ensuring a clean, user-friendly, and responsive design that enhances the overall user experience.

ii. Programming languages

Front-end tool

HTML (HyperText Markup Language): HTML is the backbone of web development, used to structure content on web pages. We have used it to defines elements like headings, paragraphs, images, links, and forms, providing the essential framework for any website or web application.

CSS (Cascading Style Sheets): CSS is a styling language that enhances the visual

presentation of web pages. We have used it to control layout, colors, fonts, animations, and responsiveness, ensuring a consistent and engaging user experience across different screen sizes and devices.

JavaScript (JS): JavaScript is a powerful programming language that brings interactivity and dynamic functionality to web applications. It is used in NagarYatra to enable real-time updates, animations, event handling, and API interactions, making web pages more engaging and responsive.

AJAX (Asynchronous JavaScript and XML): AJAX is a web development technique that enables asynchronous data exchange between the client and server without requiring a full page reload. We have used it to load web application i.e., NagarYatra by fetching and updating data in the background, improving user experience and performance. We have used AJAX to implement the simple chatbot to help the users.

Back-end tool

PHP: (Hypertext Preprocessor): PHP is a server-side scripting language used to create dynamic and interactive web applications. We have used it in NagarYatra to process requests, interact with databases, and generate dynamic content before sending it to the browser. It has helped us in building the website due to its flexibility and ease of integration with databases like MySQL.

iii. Database platform

MySQL: MySQL is a popular open-source relational database management system (RDBMS) used to store, manage, and retrieve structured data efficiently. It is used in NagarYatra to handle databases for applications, enabling secure and scalable data storage. MySQL works seamlessly with languages like PHP, so we have used it in our web application i.e., NagarYatra.

iv. Payment Gateway

eSewa: eSewa is South Asia's first and Nepal's leading online Payment Service Provider (PSP). In the NagarYatra platform, we have integrated eSewa as our primary payment gateway due to its widespread popularity, user-friendly interface, and seamless integration capabilities. Its compatibility with PHP makes the implementation process straightforward and efficient, providing users with a secure and convenient payment experience.

v. Others

Summernote: In the NagarYatra platform, we have integrated Summernote, a WYSIWYG editor, to capture and display formatted content such as driver experience and vehicle descriptions. This enables users to input rich text and ensures that the information is presented in a structured and visually appealing manner across the application.

2.3.2 Module Description

i. User Modules

a. Admin Module:

Admin module of “NagarYatra” consists of the following functions.

- Add Passenger, Driver, vehicle, vehicle_category, Vehicle_Company, Cities, Fairs
- Verify Passenger, Driver, vehicle.
- Manage Passenger, Driver, vehicle.

b. Driver Module:

Driver module of “NagarYatra” consists of the following functions.

- Add/Edit profile details.
- Add Vehicles.
- View Booking requests and accept/reject Ride.
- Mail regarding status to ride.

c. Passenger Module:

Passenger module of “NagarYatra” consists of the following functions.

- View Vehicle of that particular city on map based on search parameters.
- Send Booking request and receive status.
- View/Edit profile information.
- Provide Rating about the Ride.

2.3.3 Testing

i. Unit Testing:

We have tested the individual components such as the registration form, login functionality, booking form, OTP generation, and driver acceptance logic. Each module was tested separately to ensure it performed correctly, handled valid and invalid inputs, and produced the expected outcomes before integrating with other parts of the system.

Table 2.15: Test Case 001 - Registration form

TC-001	Description	Expected Result	Tested By	Test Result
1	Enter valid form field details	Sends email confirmation link	Kajal Kumari Mehta	Successful
2	Invalid details	Display validation error	Kajal Kumari Mehta	Successful

Table 2.16: Test Case 002 - Login form

TC-002	Description	Expected Result	Tested By	Test Result
1	Enter valid email address and password	Website should redirect to respective user page	Shibu Sharma	Successful

2	Invalid email and password	Display error message	Shibu Sharma	Successful
---	----------------------------	-----------------------	--------------	------------

Table 2.17: Test Case 003 – Algorithm

TC-003	Description	Expected Result	Tested By	Test Result
1	Search the driver and locate it.	If nearer Driver found, then locate them on map. Else, show alert dialog displaying “No nearby Driver found.”	Shibu Sharma	Successful

Table 2.18: Test Case 004 – Booking Form

TC-004	Description	Expected Result	Tested By	Test Result
1	Enter Booking form details	Website should display Booking request success message and redirect to success page.	Shibu Sharma and Kajal Kumari Mehta	Successful
2	Submitting blank form or invalid details	Display error message.	Shibu Sharma	Successful

Table 2.19: Test Case 005 – Booking Request Status

TC-005	Description	Expected Result	Tested By	Test Result
1	Accept the Booking request	-Mail booking request accepted message to passenger. -Respective booking request should be removed from the requested booking section to request confirmed section.	Shibu Sharma and Kajal Kumari Mehta	Successful
2	Reject the booking request	-Mail booking request rejected message to passenger. -Respective booking request should be removed from the requested booking section to	Shibu Sharma and Kajal Kumari Mehta	Successful

		request rejected section.		
--	--	---------------------------	--	--

Table 2.20: Test Case 006 – Reset Password (Forgot Password)

TC-003	Description	Expected Result	Tested By	Test Result
1	Enter own verified email and click ok.	Send 6 digits numeric password reset code to the email	Kajal Kumari Mehta	Successful
2	Entering any random not verified email address.	Message: User with this email does not exist. Provide your email address correctly.	Kajal Kumari Mehta	Successful

Table 2.21: Test Case 007 – Change Password

TC-003	Description	Expected Result	Tested By	Test Result
1	Enter current password(correct), new password and click change password button.	Send 6 digits numeric code to that email.	Shibu Sharma	Successful
2	Entering wrong current password and clicking change password button.	Message: Give your current password correctly. Login Unsuccessful.	Shibu Sharma	Successful

ii. System Testing:

System testing is the performance-based, end-to-end software testing of an entire system. This end-to-end testing of NagarYatra includes aspects of functional testing, non-functional testing, interface testing and recovery testing.

Table 2.22: Test Scenario 00_01- Proceeding for Booking

Test Scenario	Test-Case ID	Test Case	Test-Data	Expected Output	Actual Output	Tested By	Status
TC_00_01	TC_001	Registration	Email: Password: Confirm Password:	Success message and redirect to login page.	Success message and redirect to login page.	Shibu Sharma and Kajal Kumari Mehta	Pass
	TC_002	Login	Email: Password:	Display home page	Display home page	Kajal Kumari	Pass

				with user logged in status.	with user logged in status.	Mehta	
	TC_003	Search	Select vehicle and other details	If: nearest Drivers found locate them on map. Else: display message no nearest Drivers found	In case, search parameters matched, and nearer Drivers found, then are shown in map else message is displayed “no nearer Drivers found.”	Shibu Sharma and Kajal Kumari Mehta	Pass
	TC_004	Bookin g	Entering details for booking request	Display booking request success. Message and redirect to success page.	Displayed booking request success message and redirected to success page	Shibu Sharma	Pass

Chapter 3: Conclusion and Recommendation

3.1 Summary

Outcome: It involves the development of “NagarYatra” which will automatically finds the nearby driver and sends the request to the nearby 10Km driver and if suited Driver found than one can proceed for booking. Also get the booking status through email. In addition, users can make changes to their profile details.

Lesson learnt: During the development of NagarYatra website, we learnt how various contents need to be arranged in Websites. Its working procedure, user redirection process, designing the interactive user interface and how to handle the user’s data. It helped to implement theoretical knowledge gained from various subjects practically.

3.2 Conclusion

“NagarYatra” is developed for connecting the passengers to Vehicle that best fit their transportation needs. It involves searching for Vehicle and proceeding for booking form for getting to book and booking status to passenger is informed through mail. This website is developed using HTML, CSS, JavaScript, and PHP that ensures security, interactive user interface, focuses on validation control. This website users will be Driver, passenger, and Admin. Admin will be handling the verification process of Drivers, Vehicle and Other data.

3.3 Future Recommendation

1. Develop Mobile Application.
2. Real Time Tracking.
3. Vehicle Rental on Per day Basis

REFERENCES

- Development, D. o. (2023). *Improving urban transportation infrastructure in Nepal: Challenges and strategies*. Kathmandu: Government of Nepal. (Accessed on: 30th December,2024)
- Explore InDrive for easy, affordable rides with transparent fare prices*. (n.d.). Retrieved from indrive: <https://indrive.com/>(Accessed on: 8th January,2025)
- Nepal's No 1 Online taxi booking service provider*. (n.d.). Retrieved from Taximandu: <https://taximandu.com/>(Accessed on: 8th January,2025)
- Pathao. (n.d.). *Pathao - Rides, Food, and Parcel delivery in Nepal*. Retrieved from Pathao : <https://pathao.com/>(Accessed on: 8th January,2025)
- Rider, S. (n.d.). *Sajilo Rider – Nepal's Own Ride Sharing App*. Retrieved from Sajilo Rider: <https://sajilorider.com/>(Accessed on: 15th February,2025)
- Rides, K. (n.d.). *Kawa Rides – Nepal's Ride-Sharing App*. Retrieved from Kawa Rides: <https://kawaride.com/>(Accessed on: 15th February,2025)
- Nichat, M. R. (2015). Landmark based shortest path detection by using A* Algorithm and Haversine Formula. *ResearchGate*, 298-301. (Accessed on: 10th April,2025)

APPENDICES

Appendix 1:

```
user > book_ride.php > PHP > haversineDistance()
54 // Step 3: Calculate shortest distance
55 // 1 reference
56 function haversineDistance($lat1, $lon1, $lat2, $lon2): float
57 {
58     $earthRadius = 6371; // Radius in kilometers
59
60     $dLat = deg2rad($lat2 - $lat1);
61     $dLon = deg2rad($lon2 - $lon1);
62
63     $a = sin($dLat / 2) * sin($dLat / 2) +
64         cos(deg2rad($lat1)) * cos(deg2rad($lat2)) *
65         sin($dLon / 2) * sin($dLon / 2);
66
67     $c = 2 * atan2(sqrt($a), sqrt(1 - $a));
68     $distance = $earthRadius * $c;
69
70     return $distance;
71 }
72
73 $shortestDistance = PHP_INT_MAX;
74 $shortestVehicleId = null;
75 $shortestUserId = null;
76
77 foreach ($users as $user) {
78     $distance = haversineDistance($lat1: $pickup_lat, $lon1: $pickup_lng, $lat2: $user['latitude'], $lon2: $user['longitude']);
79
80     if ($distance < $shortestDistance) {
81         $shortestDistance = $distance;
82         $shortestVehicleId = $user['vehicle_id'];
83         $shortestUserId = $user['id'];
84     }
85 }
```

Figure 1: Implementation of Haversine Formula

Appendix 2:

```
EXPLORER: N... jister_style.css master_header.php admin_header.css connection.php M createdb.php index.php M X
> .dist
> vendor
> htaccess
> change_password.php
> cities.php
> composer.lock
> delete_user.php
> drivers.php
> fairs.php
> feedback_form.php
> feedbacks.php
> index.php M
> login.php
> logout.php
> master_footer.php
> master_header.php
> profile.php
> report.php
> update_profile.php
> users.php
> vehicles.php
> assets
> config
> css
> frontend
> oooo
main PHP Live Not Committed Yet In 60, Col 1 Spaces 4 UTF-8 CRLF () PHP Go Live 8.2.12: 8.2 Quokka Go Live Prettier

admin > index.php > script
40 <script>
61 function initMap() {
62     let defaultLocation = { lat: 26.4484334, lng: 87.2758998 }; // Fallback location
63
64     let map = new google.maps.Map(document.getElementById('map'), {
65         center: defaultLocation,
66         zoom: 100,
67         mapTypeId: 'roadmap'
68     });
69
70     // Check if @olocation is supported
71     if (navigator.geolocation) {
72         navigator.geolocation.getCurrentPosition(
73             function (position) {
74                 let userLocation = {
75                     lat: position.coords.latitude,
76                     lng: position.coords.longitude
77                 };
78
79                 // Set map center to user's current location
80                 map.setCenter(userLocation);
81
82                 // Add a marker at user's location
83                 new google.maps.Marker({
84                     position: userLocation,
85                     map: map,
86                     title: "You are here!"
87                 });
88             },
89             function (error) {
90                 // Handle error
91             }
92         );
93     }
94 }
```

Figure 2: Integration of Google Map

Appendix 3:

```
user > view_shortest_path.php > html > body > script > initMap
9 <html>
36 <body>
42 <script>
43   const pickup = { lat: parseFloat("<?=$pickup_lat ?>"), lng: parseFloat("<?=$pickup_lng ?>") };
44   const destination = { lat: parseFloat("<?=$dest_lat ?>"), lng: parseFloat("<?=$dest_lng ?>") };
45
46   function initMap() {
47     const map = new google.maps.Map(document.getElementById("map"), {
48       zoom: 14,
49       center: pickup,
50     });
51
52     const directionsService = new google.maps.DirectionsService();
53     const directionsRenderer = new google.maps.DirectionsRenderer({
54       map: map,
55       suppressMarkers: false,
56       polylineOptions: {
57         strokeColor: "#092448",
58         strokeWeight: 5,
59       }
60     });
61
62     const request = {
63       origin: pickup,
64       destination: destination,
65       travelMode: google.maps.TravelMode.DRIVING
66     };
67
68     directionsService.route(request, function(result, status) {
69       if (status === google.maps.DirectionsStatus.OK) {
```

Figure 3: View Shortest Path

SCREENSHOT'S

Appendix 4:

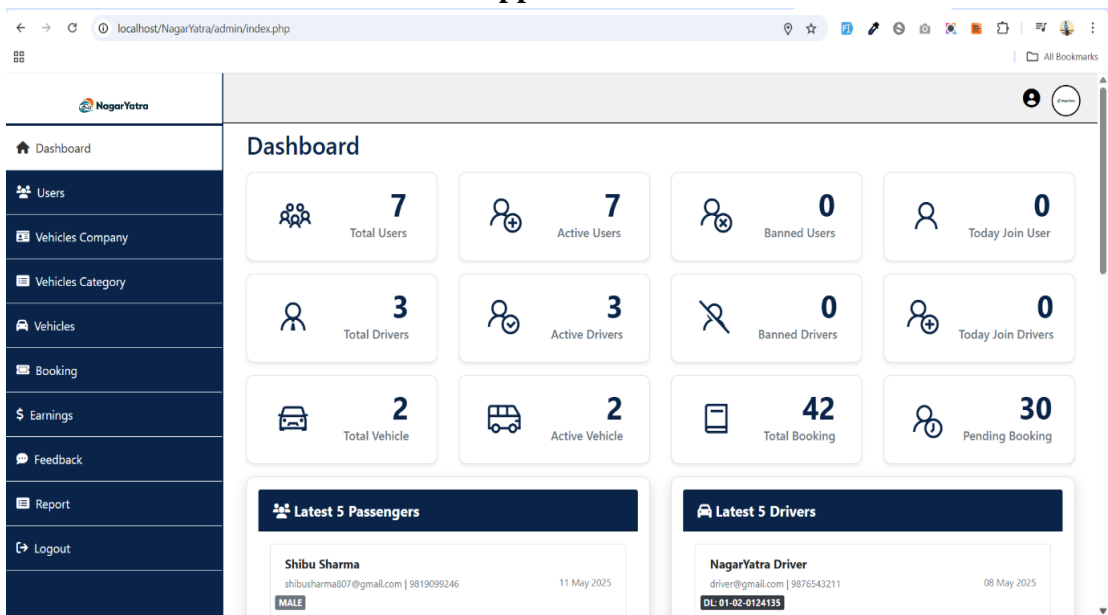


Figure 4: Admin Dashboard Interface

Appendix 5:

localhost/NagariYatra/admin/users

Users

All | All Status | Sort by Name | Search | + Add User

ID	Name	Role	Email	Phone	Status	Gender	Address	Actions
1	Aakash Deep Shah		sahakash799@gmail.com	9812345679	Active	MALE	Bargachi	
2	John		hukkachilam60@gmail.com	9819099130	Blocked	MALE	brt	
3	NagariYatra Driver		driver@gmail.com	9876543211	Active	FEMALE	Los Angeles, USA	
4	Shib558		shibusharma8077@gmail.com	9819099126	Active	MALE	Rampur	
5	Shibu sharma		ss02001135@gmail.com	9819079126	Active	MALE	Rampur	
6	Shibu sharma		chaudharyshubhanshu@gmail.com	9819099126	Active	MALE	Rampur	
7	Shibu sharma		hukkachilam610@gmail.com	9819099187	Active	FEMALE	Rampur	
8	Shibu Sharma		shibusharma807@gmail.com	9819099246	Active	MALE	Brt - 6,Rampur	
9	Unknown		s@gmail.com	9812345678	Active	MALE	ad	
10	User		user@gmail.com	9812345678	Active	MALE	Rampur	

Figure 5: User list for Admin

Appendix 6:

localhost/NagariYatra/admin/booking

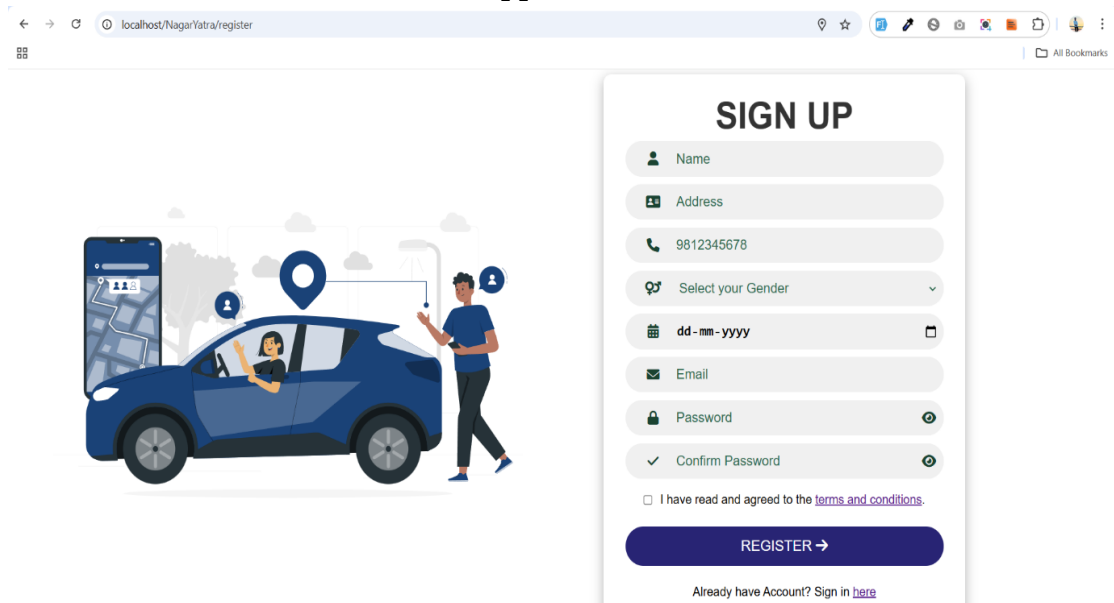
Bookings

Search by pickup or destination | dd-mm-yyyy | dd-mm-yyyy | All Status

S.N	Pickup	Destination	Date	Booking Status	Action
1	F561+X4 Amahibelha, Nepal	F8HP+JR Katahari, Nepal	14 May 2025	Completed	
2	F7G8+HQM, Biratnagar 56613, Nepal	F8CC+X7 Katahari, Nepal	14 May 2025	Accepted	
3	F5CQ+3X2 Eastern Sugar Mill Private Limited, Amahibelha, Nepal	F8RV+XR Bhaudaha, Nepal	14 May 2025	Completed	
4	F5CQ+3X2 Eastern Sugar Mill Private Limited, Amahibelha, Nepal	F8VR+W9 Bhaudaha, Nepal	14 May 2025	Completed	

Figure 6: Booking list for Admin

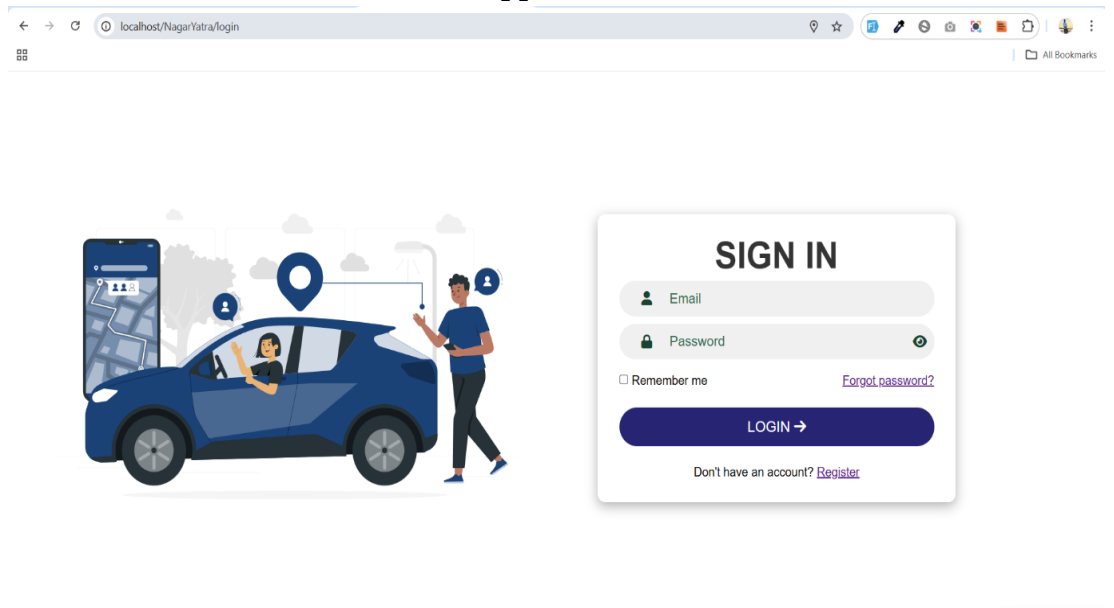
Appendix 7:



The screenshot shows a web browser at the URL `localhost/NagarYatra/register`. On the left is an illustration of a blue car with a person inside, a person standing outside, and a smartphone displaying a map. On the right is a 'SIGN UP' form with the following fields: Name, Address, Phone number (pre-filled with 9812345678), Gender (dropdown), Date of birth (dd-mm-yyyy), Email, Password, and Confirm Password. There is a checkbox for 'I have read and agreed to the [terms and conditions](#)'. A purple 'REGISTER →' button is at the bottom, followed by the text 'Already have Account? Sign in [here](#)'.

Figure 7: Registration Form for Users

Appendix 8:



The screenshot shows a web browser at the URL `localhost/NagarYatra/login`. On the left is the same illustration as in Figure 7. On the right is a 'SIGN IN' form with the following fields: Email and Password. There is a checkbox for 'Remember me' and a link for '[Forgot password?](#)'. A purple 'LOGIN →' button is at the bottom, followed by the text 'Don't have an account? [Register](#)'.

Figure 8: Login Form for user

Appendix 9:

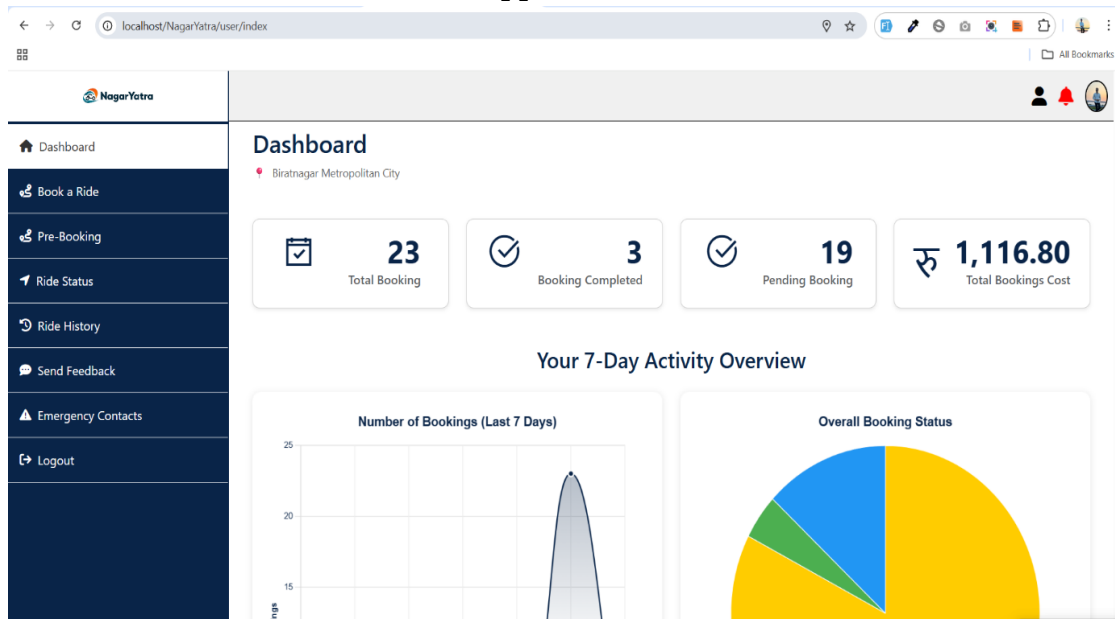


Figure 9: User Dashboard (Passenger)

Appendix 10:

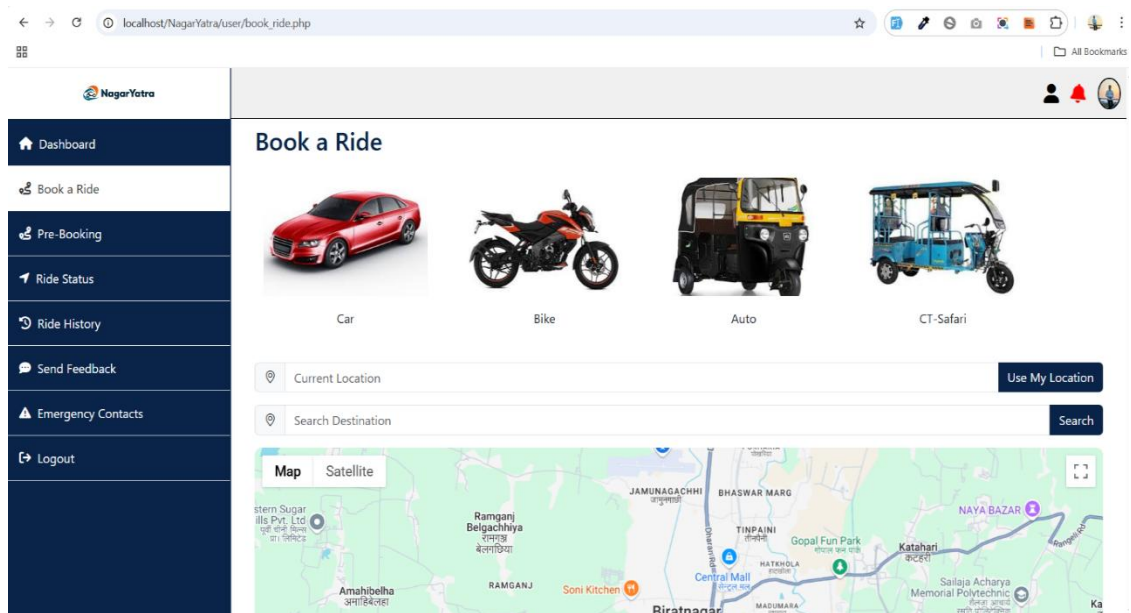


Figure 10: Booking Interface of Passenger

Appendix 11:

NagarYatra

Dashboard

Book a Ride

Pre-Booking

Ride Status

Ride History

Send Feedback

Emergency Contacts

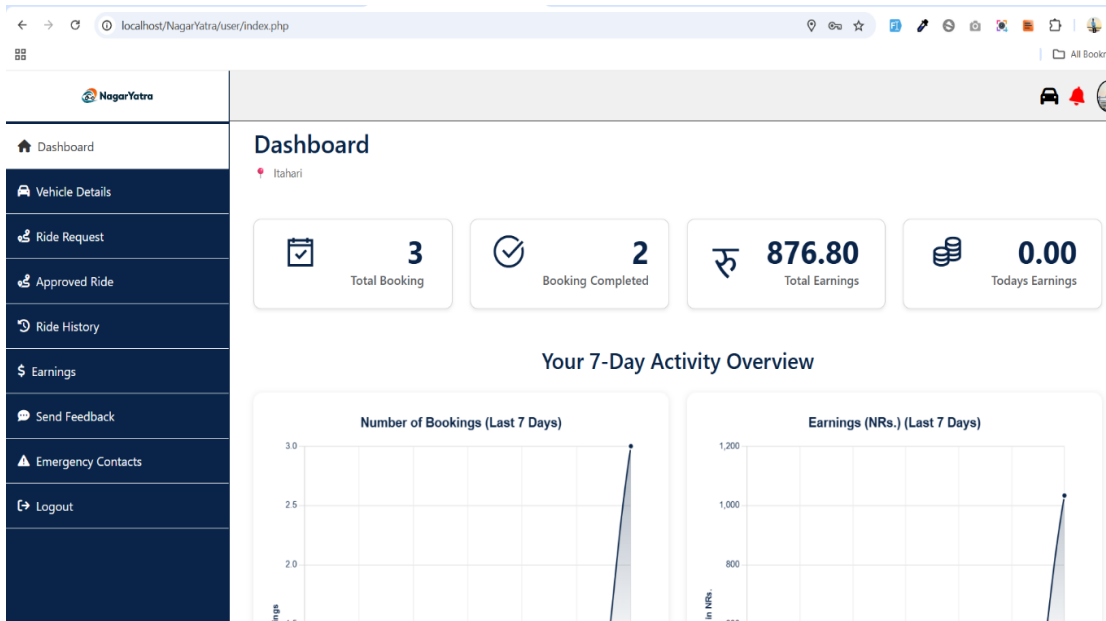
Logout

Ride Status

SN	Pickup	Destination	Cost	Duration	Date	Map	Status	Action
1	View Map	View Map	Rs. 24.76	8	2025-05-14 14:13:22	View Shortest Path	Pending	Cancel
2	F7G8+HQM, Biratnagar 56613, Nepal View Map	F6RR+3GW, Purba Kushaha 56613, Nepal View Map	Rs. 39.90	7	2025-05-14 12:28:46	View Shortest Path	Pending	Cancel
3	F79J+4H8, Biratnagar 56613, Nepal View Map	F6CH+FF, Ramganj Belgachhiya 56613, Nepal View Map	Rs. 111.88	6	2025-05-14 12:28:09	View Shortest Path	Pending	Cancel
4	F7G8+HQM, Biratnagar 56613, Nepal View Map	F6VJ+X6 Ramganj Belgachhiya, Nepal View Map	Rs. 60.40	14	2025-05-14 12:27:37	View Shortest Path	Pending	Cancel
5	F7G8+HQM, Biratnagar 56613, Nepal View Map	F6R6+8Q Purba Kushaha, Nepal View Map	Rs. 97.70	14	2025-05-14 12:27:15	View Shortest Path	Pending	Cancel
6	F7F6+WGI, Biratnagar	F792+H6, Biratnagar	Rs.		2025-05-	View		

Figure 11: Ride Status of Passengers

Appendix 12:



Appendix 13:

The screenshot shows the 'Approved Rides' interface of the NagarYatra app. The sidebar on the left contains the following navigation options: Dashboard, Vehicle Details, Ride Request, Approved Ride (selected), Ride History, Earnings, Send Feedback, Emergency Contacts, and Logout. The main content area displays a table of approved rides.

S.N	Pickup Location	Destination Location	Estimated Km	Estimated Duration	Estimated Cost	OTP	Map	Action
1	F7G8+HQM, Biratnagar 56613, Nepal	F8CC+X7 Katahari, Nepal	9.66	29	156.88	650135	View Shortest Path	Ride Complete Cancel

Figure 13: Approved Rides Interface of Driver

Appendix 14:

The screenshot shows the eSewa payment gateway integration. The interface is divided into two main sections: a payment summary on the left and a login/sign-in form on the right.

Payment Summary (Left):

- EPAYTEST
- Total Amount: NPR. 110.00
- Total Amount: 110.0
- Discount: 15% (on food & cakes)

Login/Sign-in Form (Right):

- Sign in to your account
- Phone Number: 9806800001
- Password: ****
- I'm not a robot (Captcha)
- LOGIN button
- Forgot Password?
- Don't have an account? Register
- CANCEL PAYMENT button

Figure 14: Integration of eSewa as Payment Gateway